HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

FINAL PROJECT REPORT

Topic: Parallelization Methods for CNN-Based Computer Vision Inference

Course: Parallel and Distributed Programming

Course code: IT4130E

Class/session: 168069 - Friday morning, periods 1–4, C7-215

Lecturer: Nguyen Tuan Dung

Group members: Luu Hieu An - 202400093
Nguyen Phu An - 20235466
Pham Chi Bang - 20235477
Nguyen Thanh Lam - 20235518

Hanoi, 2026

Contents

Abstract

This report drafts the final structure for the project *Parallelization Methods for CNN-Based Computer Vision Inference*. The repository `yolo-mpi-people-count` contains two complementary directions. Method 1 is an implemented YOLO/OpenMPI video people-counting pipeline that parallelizes inference at task level over video frames and image tiles. Method 2 is a VGG11 convolution benchmark that is designed to demonstrate feature-map data parallelism with 2D block decomposition and halo exchange.

For Method 1, the report uses measured values already recorded in the project reports. The YOLO pipeline runs on a three-MacBook physical cluster with C++17 and OpenMPI. A video is decomposed temporally into frames and spatially into tiles; each frame-tile pair becomes an MPI task. The implementation supports static block-cyclic scheduling, dynamic master-worker scheduling, rank-local YOLO workers, coordinate remapping, duplicate removal, global NMS, CSV output, and per-rank timing.

The recorded YOLO experiments verify serial-versus-MPI correctness on 30 frames with zero mismatched frames. The detector accuracy experiment on 300 MOT17-derived frames reports mean absolute count error 7.983 and predicted average count 8.223 versus ground-truth average 16.207. The input-size experiment selects $N = 600$ frames because the 12-process runtime is 123.667 seconds, satisfying the required two-to-three-minute workload. On $2N = 1200$ frames, the 12-process wall-clock speedup is 1.939x in the original sweep, while two-repeat follow-up measurements report mean 12-process runtime 178.174 seconds and mean speedup 1.781x. Static scheduling outperforms dynamic scheduling on the measured 600-frame 5x4 workload, and weighted static placement improves the idle-gap indicator from 0.466 to 0.235.

For Method 2, the repository contains implementation notes and run scripts for VGG11 convolution experiments, but the checked-out project does not include completed CSV result files. Therefore, the report includes a complete draft framework and placeholder tables for correctness, input size, granularity, speedup, and blocking versus non-blocking halo communication. These tables are intentionally left for final measured values rather than fabricated numbers.

Chapter 1

Introduction

Computer vision inference is a useful setting for studying parallel programming because the workload is computationally heavy, data-rich, and naturally decomposable. A CNN-based detector or classifier can be applied repeatedly across frames, image regions, batches, or feature-map blocks. These repeated operations expose the main concerns of a parallel system: decomposition, mapping, communication, load balancing, correctness, and speedup.

This project focuses on CNN-based inference rather than model training. The main implemented application is video people counting with YOLO. Given an input video, the system predicts people bounding boxes and frame-level counts. The expensive work is detector inference over many frames and image tiles. The parallel contribution is the C++17/OpenMPI runtime around the detector: it creates tasks, schedules them across MPI ranks, launches local detector workers, gathers detections, removes duplicates, and writes experiment artifacts.

The repository also contains a second method for CNN convolution parallelization using VGG11 without BatchNorm. This method is structurally different from the YOLO pipeline. Instead of assigning independent frame-tile inference tasks, it decomposes convolution feature maps into 2D blocks, exchanges halo data between neighboring ranks, and compares blocking and non-blocking MPI communication. The checked-out repository contains the implementation and runbook for this direction, but it does not include final measured CSV outputs. Therefore, this report treats Method 2 as a complete framework with pending result tables.

The report separates two types of correctness. Model accuracy means whether YOLO counts match human ground truth from MOT17. Parallel correctness means whether the MPI implementation produces the same result as the serial version of the same pipeline. A detector can undercount in crowded scenes while the parallel implementation remains correct. The measured results from the repository show this distinction clearly: serial and MPI counts match in the correctness test, while detector-versus-ground-truth accuracy remains limited on crowded MOT17 frames.

The main objectives are:

- define CNN-based computer vision inference as a parallel workload;
- present Method 1: task parallelism for YOLO video inference;
- present Method 2: data parallelism for VGG11 convolution layers;
- report measured YOLO correctness, input-size, scheduling, granularity, speedup, and weighted-placement results from the repository;
- leave explicit placeholder tables for experiments that still require final CSV evidence.

Chapter 2

Problem Description

2.1 CNN-Based Computer Vision Inference

The shared problem is to execute CNN-based inference faster by distributing work across MPI processes. Let the input be either a video sequence or a CNN feature tensor. In both cases, the program applies a fixed pretrained model; it does not train or update model weights. The parallel system must preserve the output of the serial inference pipeline while reducing runtime or revealing the cost of communication.

2.2 Method 1: YOLO Video People Counting

For the YOLO method, the input video is a finite sequence of frames:

$$V = \{F_1, F_2, \ldots, F_N\}. \tag{2.1}$$

The target output for each frame is the number of detected people after post-processing:

$$count(F_i) = |\{b \mid b \text{ is a final person bounding box in } F_i\}|. \tag{2.2}$$

The detector backend is a pretrained YOLO model through a local Python worker process [2]. The MPI program does not split YOLO neural-network kernels. Instead, MPI distributes repeated YOLO calls over frame-tile tasks.

For a tile grid with R rows and C columns, frame F_i is decomposed into:

$$F_i \to \{T_{i,1}, T_{i,2}, \ldots, T_{i,R\times C}\}. \tag{2.3}$$

A task is one frame-tile inference:

$$task(i, j) = T_{i,j}, \qquad |\mathcal{T}| = N \times R \times C. \tag{2.4}$$

Rank 0 converts tile-local detections back to full-frame coordinates, applies tile ownership filtering, removes duplicates, and writes final frame counts.

2.3 Method 2: VGG11 Convolution Benchmark

For the VGG11 method, the input is a CNN feature map. Each convolution layer receives a tensor and computes an output tensor using a fixed 3×3 kernel. The repository uses VGG11 without BatchNorm, with scaled channel profiles such as `tiny`, `small`, and `full`. The purpose is to demonstrate data parallelism inside CNN convolution layers.

For each convolution layer, P MPI ranks are arranged into a $P_r \times P_c$ process grid. The spatial dimensions $H \times W$ of the feature map are split into 2D blocks. Each rank computes the convolution output for its local core block. Since a 3×3 convolution needs neighboring pixels, ranks must exchange one-pixel halo rows, columns, and corners before local computation.

Method 1 and Method 2 therefore exercise different parallel-programming patterns:

Table 2.1: Comparison of the two project methods

Aspect	Method 1: YOLO video inference	Method 2: VGG11 convolution
Parallelism level	Task parallelism	Data parallelism
Decomposition	Temporal-spatial frame/tile tasks	2D feature-map blocks
Mapping	1D flattened task mapping	2D block/process-grid mapping
Communication	Master-worker dispatch and result collection	Neighbor halo exchange
Topology	Star topology around rank 0	2D Cartesian/mesh-like topology
Correctness target	Serial count equals MPI count	Serial convolution/VGG stack equals distributed output

2.4 Correctness Targets

The serial YOLO baseline processes the same frame-tile task list without MPI distribution and applies the same post-processing pipeline. Parallel correctness is:

$$\Delta_i = |count_{parallel}(F_i) - count_{serial}(F_i)|. \tag{2.5}$$

The VGG11 correctness target is numerical agreement between the serial convolution stack and the distributed convolution stack, typically reported as maximum absolute error or pass/fail under a chosen tolerance.

Chapter 3

Parallelization Methods

3.1 Method 1: Task Parallel YOLO Inference

The YOLO pipeline uses task-level parallelism over independent frame-tile inference tasks. The implementation is centered on `src/yolo_mpi_cpp.cpp` in the repository. Each MPI rank runs the same C++ program, and rank-local detector work is delegated to a long-lived Python YOLO worker.

3.1.1 Temporal-Spatial Decomposition

The input video is first decomposed by time into frames and then by space into image tiles. If the video has N frames and each frame is split into an $R \times C$ grid, the total number of tasks is NRC. Larger grids create more tasks and can improve scheduling flexibility, but they increase duplicate detections near tile borders and increase post-processing work.

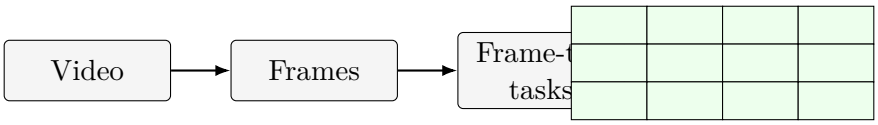

Figure 3.1: Hybrid temporal-spatial decomposition for YOLO video inference.

3.1.2 1D Mapping and Static Scheduling

All frame-tile tasks are flattened into a one-dimensional list. For task index i, chunk size k, and P ranks, static block-cyclic scheduling assigns:

$$owner(task_i) = \left\lfloor \frac{i}{k} \right\rfloor \bmod P. \tag{3.1}$$

Static scheduling has low dispatch overhead because ranks can determine their own tasks locally. The measured repository results show that this advantage matters for the 600-frame 5x4 tiled YOLO workload.

3.1.3 Dynamic Master-Worker Scheduling

Dynamic scheduling uses rank 0 as a master. The master owns the remaining task queue, dispatches tasks to workers, receives completed result payloads, and sends either the next task or a stop message. If `master_compute=1`, rank 0 may also process local tasks while waiting. The dynamic scheduler uses `MPI_Iprobe` for polling readiness, while payload transfer still uses blocking `MPI_Send` and `MPI_Recv`.

3.1.4 Communication and Merging

The dynamic topology is a star centered on rank 0. Workers do not exchange detections with one another. Rank 0 gathers detections, remaps tile-local boxes into frame coordinates, applies tile-owner filtering, runs global non-maximum suppression, and computes per-frame counts. This centralized merge is necessary because people near tile boundaries may be detected by multiple tiles.

3.1.5 YOLO Algorithm Sketch

Algorithm 1 Static MPI YOLO people-counting pipeline

Build task list $\mathcal{T}$ from N frames and an $R \times C$ tile grid.
for each MPI rank r in parallel **do**
    Select tasks whose static owner is r.
    Run local YOLO inference for selected tasks.
    Store detections and timing metrics.
end for
Gather serialized detections and rank metrics to rank 0.
Rank 0 remaps boxes, removes duplicates, computes frame counts, and writes CSV outputs.

Algorithm 2 Dynamic master-worker YOLO people-counting pipeline

if rank is 0 **then**
    Initialize task queue and dispatch initial tasks to available workers.
    while unfinished tasks or active workers remain **do**
        Poll for ready worker results with `MPI_Iprobe`.
        Receive completed payloads using blocking `MPI_Recv`.
        Send next task or stop message using `MPI_Send`.
        Optionally compute one local task if `master_compute` is enabled.
    end while
    Merge detections and write output tables.
else
    while a task is received **do**
        Run local YOLO inference and send results to rank 0.
    end while
end if

3.2 Method 2: Data Parallel VGG11 Convolution

The VGG11 method targets a finer-grained CNN operation. It parallelizes the spatial dimensions of convolution feature maps rather than independent video tasks. This method is useful because it demonstrates a different communication pattern: neighbor exchange rather than master-worker collection.

3.2.1 2D Feature-Map Decomposition

For a feature map of height H and width W, ranks are arranged into a $P_r \times P_c$ grid. Each rank owns a rectangular core block. The 2D mapping is row-major, for example a 3x4 grid with 12 ranks.

3.2.2 Halo Exchange

A 3×3 convolution requires one cell of neighboring context. Before computing a local block, each rank exchanges boundary rows, columns, and corners with its neighboring ranks. Blocking communication can implement this with ordered send/receive phases. Non-blocking communication can post receives and sends first, then wait for completion, allowing possible overlap and reducing deadlock risk.

3.2.3 VGG11 Algorithm Sketch

Algorithm 3 Distributed VGG11 convolution layer

1: Arrange P ranks into a $P_r \times P_c$ process grid.
2: Rank 0 splits the input feature map into 2D core blocks.
3: Scatter core blocks to ranks.
4: Exchange one-pixel halo rows, columns, and corners with neighboring ranks.
5: Each rank computes convolution on its local core output.
6: Gather local output blocks to rank 0.
7: Rank 0 applies ReLU and MaxPool before the next layer.

3.3 Load Balancing and Metrics

Both methods record computation, communication, and idle time. The report uses:

$$T_r = T_{compute,r} + T_{comm,r} + T_{idle,r}, \tag{3.2}$$

and the idle-gap indicator reported by the project scripts. The course criterion treats the run as balanced when the idle gap is at most 0.25. The measured YOLO results show that equal process placement does not meet this criterion, while the best weighted static placement does.

Chapter 4

Experimental Setup

4.1 Repository Evidence

The draft is based on the local repository `yolo-mpi-people-count`. The strongest source for measured YOLO values is `reports/final_report_hust_style.md`, with additional full-cluster discussion in `reports/additional_experiments_20260623.md`. The checkout does not include the raw CSV result directories referenced by those reports, so this document records the published report values and leaves missing experiment tables as pending.

4.2 Physical Cluster

The project uses three physical MacBook machines connected through a local network. The benchmark mode uses CPU execution to match the course emphasis on MPI processes and processor assignment. GPU/MPS execution is kept for demonstration and live-camera use rather than formal CPU speedup measurement.

Table 4.1: Physical cluster roles

Host	Machine type	Role in experiments
master	MacBook Air M4	Coordinates execution, gathers results, stores outputs, and can run local tasks.
node1	MacBook Pro M4	Worker host; measured results suggest it can sustain more assigned ranks.
node2	MacBook Air M2	Worker host; weighted placement assigns fewer ranks to reduce imbalance.

4.3 Software and Data

The implementation uses C++17, OpenMPI, Python helper scripts, and a pretrained Ultralytics YOLO detector. The main dataset source is MOT17-derived video data [1]. Compact MOT17 subsets are used for quick correctness and granularity experiments; longer sequence segments are used for the required input-size and speedup runs.

Table 4.2: Main YOLO experimental configuration

Item	Setting
Detector	Pretrained YOLO, person-class filtering
Parallel framework	C++17 and OpenMPI
Benchmark device	CPU
Dataset	MOT17-derived sequences
Main process count	12 MPI processes
Main long-run grid	5x4 regions
Schedulers	Static block-cyclic and dynamic master-worker
Placement variants	Uniform 4/4/4 and weighted 3/6/3, 4/6/2

4.4 Metrics

The report uses the following metrics from the project reports and generated CSV design:

- serial-versus-MPI correctness: mismatched frames and count error;
- detector accuracy: MAE, RMSE, percentage error, exact-match ratio;
- runtime with communication: wall-clock time including coordination;
- runtime without communication: compute-oriented time reported by the scripts;
- speedup and efficiency:

$$S_p = \frac{T_1}{T_p}, \qquad E_p = \frac{S_p}{p}; \tag{4.1}$$

- idle-gap indicator for load-balance analysis.

4.5 Experiment Matrix

Table 4.3: Experiment matrix for final report completion

Experiment	Method	Status in this draft	Report purpose
Parallel correctness	YOLO	Filled from report values	Verify MPI output against serial baseline.
Detector accuracy	YOLO	Filled from report values	Separate model accuracy from parallel correctness.
Input-size selection	YOLO	Filled from report values	Choose N near 120–180 s.
Granularity	YOLO	Filled from report values	Study task count, communication, and idle time.
Scheduling	YOLO	Filled from report values	Compare static and dynamic scheduling.
Speedup	YOLO	Filled from report values	Measure S_p and E_p.
Weighted placement	YOLO	Filled from report values	Address heterogeneous physical cluster.
Correctness	VGG11	Pending CSV	Verify distributed convolution stack.
Input size and speedup	VGG11	Pending CSV	Evaluate data-parallel CNN convolution.
Blocking vs non-blocking	VGG11	Pending CSV	Compare halo exchange strategies.

Chapter 5

Results and Discussion

5.1 YOLO Parallel Correctness

Table 5.1 reports the serial-versus-MPI correctness result recorded in the repository report. The MPI run produced the same frame counts as the serial baseline on the tested sequence.

Table 5.1: YOLO serial-versus-MPI correctness

Result	Frames	Mismatched frames	Max count error	Mean error	Evidence
Passed	30	0	0	0.000	repository report

This validates the parallel pipeline for the tested configuration. It indicates that task splitting, MPI result collection, coordinate remapping, duplicate removal, and final counting preserve the serial pipeline output.

5.2 Detector Accuracy Against Ground Truth

Table 5.2 compares YOLO predicted frame counts against MOT17 ground-truth counts. This is a model-accuracy measurement, not a parallel-correctness measurement.

Table 5.2: YOLO count accuracy against MOT17 ground truth

Frames	MAE	RMSE	Percent error	Exact match	GT avg.	Pred. avg.
300	7.983	8.269	0.490	0.000	16.207	8.223

The detector undercounts crowded MOT17 scenes. This limitation is expected for a pretrained detector running without dataset-specific fine-tuning. The important report distinction is that MPI correctness passes even though detector-versus-ground-truth accuracy is imperfect.

5.3 Input-Size Selection

The compact MOT17 subset is useful for quick runs but does not reach the required two-to-three-minute runtime. The long-sequence experiment in Table 5.3 selects $N = 600$ frames because the 12-process wall-clock runtime is 123.667 seconds.

Table 5.3: YOLO input-size selection

Frames	P	Grid	Runtime with comm. (s)	Runtime without comm. (s)	Selection note
30	12	4x3	9.165	2.999	Compact quick test
60	12	4x3	12.570	6.801	Compact quick test
100	12	4x3	17.776	11.596	Compact quick test
150	12	4x3	23.585	17.103	Compact quick test
300	12	5x4	54.764	49.156	Long sequence
600	12	5x4	123.667	114.352	Selected N
837	12	5x4	146.316	139.136	Longer validation

5.4 Granularity and Load Balance

Table 5.4 summarizes the granularity experiment from the repository report. Increasing the number of regions increases task count, communication, and idle time. None of these compact-dataset runs satisfy the 25 percent idle-gap balance criterion.

Table 5.4: YOLO granularity and load balance

Grid	P	Tasks	Max comp. (s)	Avg comp. (s)	Total comm. (s)	Idle gap	Balance
1x1	12	150	1.367	0.483	47.417	0.838	Not balanced
2x2	12	600	5.447	2.205	72.434	0.802	Not balanced
4x3	12	1800	16.329	6.375	154.944	0.809	Not balanced
5x4	12	3000	25.263	9.133	230.059	0.824	Not balanced

The result shows that simply making tasks finer is not enough. Finer tiles create more scheduling opportunities, but they also increase intermediate detections, communication, and master-side merging. This motivates the scheduler and weighted-placement experiments.

5.5 Static vs Dynamic Scheduling

The full-cluster scheduler comparison used 12 processes, 600 frames, a 5x4 grid, and CPU execution. Static scheduling is much faster on this measured workload.

Table 5.5: Static versus dynamic scheduling on 600 frames

Scheduler	P	Grid	Runtime with comm. (s)	Runtime without comm. (s)	Imbalance	Idle gap	Balance
Static	12	5x4	40.619	36.522	1.202	0.466	Not balanced
Dynamic	12	5x4	102.316	96.959	2.778	0.837	Not balanced

Dynamic scheduling is conceptually useful for irregular workloads, but this result shows it is not automatically faster. For the 600-frame tiled YOLO workload, the cost of dispatching many small tasks and collecting results at rank 0 outweighs the load-balancing benefit. Static scheduling is therefore used for the final weighted-placement experiment.

5.6 Speedup and Efficiency

The original speedup experiment uses the selected input size doubled to $2N = 1200$ frames. Table 5.6 reports the recorded speedup values.

Table 5.6: YOLO speedup on 1200 frames

P	Runtime with comm. (s)	Runtime without comm. (s)	Wall speedup	Efficiency	Note
1	345.677	342.467	1.000	1.000	Baseline
2	276.155	272.546	1.252	0.626	Partial parallelism
4	244.747	240.905	1.412	0.353	More ranks
8	200.770	194.282	1.722	0.215	Multi-machine spread
12	178.306	170.255	1.939	0.162	Best in sweep

The speedup is measurable but not linear. At 12 processes, wall-clock speedup is 1.939x and computation-only speedup is approximately 2.011x. The gap comes from communication, scheduling, result collection, duplicate removal, and CPU contention.

The follow-up report also records two complete 1200-frame repeats. Those values are useful for defense because they show stable 12-process runtime but a variable one-process baseline:

Table 5.7: Two-repeat speedup summary on 1200 frames

P	Repeats	Mean runtime with comm. (s)	Runtime stdev (s)	Mean speedup	Note
1	2	317.314	40.111	1.000	Baseline varied strongly
2	2	297.950	30.823	1.078	Small gain
4	2	271.548	37.904	1.190	Moderate gain
8	2	216.715	22.549	1.482	Better cluster use
12	2	178.174	0.186	1.781	Very stable 12-rank time

5.7 Weighted Static Placement

The equal 4/4/4 placement does not meet the 25 percent idle-gap criterion. The best measured weighted placement assigns four ranks to the master, six to node1, and two to node2.

Table 5.8: Weighted static placement on heterogeneous cluster

Placement	Distribution	Runtime with comm. (s)	Runtime without comm. (s)	Idle gap	Balance
Uniform	4/4/4	40.619	36.522	0.466	Not balanced
Weighted	3/6/3	34.712	30.339	0.371	Not balanced
Weighted	4/6/2	29.581	27.484	0.235	Balanced

This result supports the claim that processor assignment matters on a heterogeneous physical cluster. Equal rank placement gives every host the same number of processes, but the machines do not have equal throughput. Weighted placement reduces runtime and satisfies the load-balance criterion.

5.8 Local-Only and Full-Cluster Observation

Additional experiments show that increasing process count on a single machine can saturate local resources. On the master-only 600-frame 5x4 run, 2 processes achieved 288.494 seconds while 4 and 8 processes were slower. In contrast, the full three-machine 600-frame dynamic sweep improved from 202.715 seconds at 1 process to 90.900 seconds at 12 processes. This supports the defense point that physical distribution matters more than only increasing the number of local MPI ranks.

5.9 VGG11 Method 2 Pending Tables

The VGG11 convolution method still needs final measured CSV values in this checkout. The following tables are intentionally left as a fill-in framework.

Table 5.9: Pending VGG11 correctness table

Profile	P	Grid	Halo mode	Max error	Result/evidence
Pending	Pending	Pending	Blocking	Pending CSV	Pending `summary.csv`
Pending	Pending	Pending	Non-blocking	Pending CSV	Pending `summary.csv`

Table 5.10: Pending VGG11 speedup and communication table

P	Input size	Halo mode	Runtime (s)	Halo comm. (s)	Speedup	Evidence
1	Pending	Blocking	Pending CSV	Pending CSV	1.000	Pending
2	Pending	Blocking	Pending CSV	Pending CSV	Pending CSV	Pending
4	Pending	Blocking	Pending CSV	Pending CSV	Pending CSV	Pending
8	Pending	Blocking	Pending CSV	Pending CSV	Pending CSV	Pending
12	Pending	Blocking	Pending CSV	Pending CSV	Pending CSV	Pending
12	Pending	Non-blocking	Pending CSV	Pending CSV	Pending CSV	Pending

5.10 Summary of Findings

The measured YOLO results support the main parallel-programming narrative. The implementation preserves serial output, reaches the required input-size runtime, shows measurable speedup, and demonstrates that scheduling and rank placement strongly affect performance. The strongest measured improvement in load balance comes from weighted static placement, which reduces the idle-gap indicator from 0.466 to 0.235. Method 2 is ready as a report framework but still requires final CSV evidence before numerical conclusions should be written.

Chapter 6

Conclusion

This report drafts the final content for *Parallelization Methods for CNN-Based Computer Vision Inference* using evidence from the `yolo-mpi-people-count` repository. The completed YOLO method demonstrates task-level parallelism with hybrid temporal-spatial decomposition, 1D task mapping, static and dynamic scheduling, master-worker communication, rank-local detector execution, and centralized detection merging.

The measured YOLO results are strong enough for the main experimental narrative. The serial-versus-MPI correctness test passes with zero mismatched frames. The selected 600-frame workload runs in 123.667 seconds with 12 processes, satisfying the required two-to-three-minute range. The 1200-frame speedup sweep reaches 1.939x wall-clock speedup at 12 processes in the original report, and follow-up repeats show a stable 12-process runtime around 178 seconds. Static scheduling outperforms dynamic scheduling on the measured 600-frame 5x4 workload, while weighted static placement improves the idle-gap indicator to 0.235 and satisfies the 25 percent load-balance criterion.

The detector accuracy result should be interpreted separately from parallel correctness. YOLO undercounts crowded MOT17 frames, with MAE 7.983 on the recorded 300-frame accuracy experiment. This is a model limitation, not evidence that MPI changed the pipeline output.

The VGG11 method provides a second, conceptually different parallelization approach: feature-map data parallelism with 2D block mapping and halo exchange. The repository contains the method design and run scripts, but no final checked-out CSV results. Its result tables remain pending and should be filled only after running the VGG11 report suite.

Overall, the project covers the core course concepts: decomposition, mapping, communication topology, scheduling, load balancing, correctness verification, input-size selection, and speedup measurement. The final work before submission is to fill any remaining pending VGG11 tables, attach raw result evidence where required, and decide whether the final PDF should emphasize YOLO as the completed method or present VGG11 as a second method with pending experiments.

Chapter 7

Member Contributions

Table 7.1 records a draft responsibility split based on the current repository contents. The group should update it with confirmed final work before submission.

Table 7.1: Draft member contributions

Member	Student ID	Contribution
Luu Hieu An	202400093	Report integration, repository evidence audit, final presentation alignment, and result table preparation.
Nguyen Phu An	20235466	Cluster setup, OpenMPI execution, hardware evidence collection, and hostfile/placement experiments.
Pham Chi Bang	20235477	C++/MPI implementation, YOLO scheduling pipeline, VGG11 convolution benchmark, and result aggregation support.
Nguyen Thanh Lam	20235518	Plot generation, speedup table preparation, experimental discussion, and final report polishing.

Bibliography

[1] MOTChallenge. Mot17 benchmark. https://motchallenge.net/data/MOT17/, 2026.

[2] Ultralytics. Ultralytics yolo documentation. https://docs.ultralytics.com/, 2026.